

Xử Lý Chữ Số & Toán Số Học Đơn Giản

Chương 1 • Bài 7 • Khóa Học Lập Trình Thi Đấu Cơ Bản THPT

NỘI DUNG BÀI HỌC

- Phần 1: Kỹ thuật tách, đếm và tính tổng chữ số của số nguyên N.
- Phần 2: Số thuận nghịch (Palindrome Number).
- Phần 3: Thuật toán kiểm tra số nguyên tố tối ưu $O(\sqrt{N})$.
- Phần 4: Sàng nguyên tố Eratosthenes cho $N \leq 10^7$.
- Phần 5: Số chính phương, Ước số và Bội chung nhỏ nhất (LCM).

1. Kỹ thuật tách chữ số cơ bản

Để tách từng chữ số của số nguyên dương N từ phải sang trái, ta sử dụng vòng lặp while ($n > 0$) với hai phép toán cơ bản:

- $digit = n \% 10$; — Lấy chữ số hàng đơn vị cuối cùng.
- $n = n / 10$; — Cắt bỏ chữ số cuối vừa lấy.

[C++]

```
long long n;  
cin >> n;  
n = abs(n); // Luôn lấy trị tuyệt đối nếu n có thể âm!  
  
int count_digits = 0;  
long long sum_digits = 0;  
  
if (n == 0) {  
    count_digits = 1;  
    sum_digits = 0;  
} else {  
    while (n > 0) {  
        int d = n % 10;  
        sum_digits += d;  
        count_digits++;  
        n /= 10;  
    }  
}  
  
cout << "Số chu số: " << count_digits << ", Tổng: " << sum_digits << "\n";
```

2. Kiểm tra số nguyên tố tối ưu $O(\sqrt{N})$

Số nguyên tố là số nguyên lớn hơn 1 chỉ có đúng 2 ước dương là 1 và chính nó. Nếu N là hợp số, nó luôn có ít nhất một ước d thỏa mãn $2 \leq d \leq \sqrt{N}$. Do đó ta chỉ cần duyệt kiểm tra tới $\sqrt{N}$.

[C++]

```
bool isPrime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}
```

3. Sàng nguyên tố Eratosthenes

Khi cần kiểm tra nhiều số hoặc tìm tất cả các số nguyên tố nhỏ hơn hoặc bằng M ($M \leq 10^7$), Sàng Eratosthenes cho phép xử lý trong thời gian cực nhanh $O(M \log(\log M))$ và trả lời mỗi truy vấn trong $O(1)$.

[C++]

```
const int MAXN = 10000000;
bool is_prime[MAXN + 1];

void sieve() {
    fill(is_prime, is_prime + MAXN + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p <= MAXN; p++) {
        if (is_prime[p]) {
            for (int i = p * p; i <= MAXN; i += p) {
                is_prime[i] = false;
            }
        }
    }
}
```

MẸO TOÁN HỌC TRÁNH SAI SỐ VÀ TRÀN BỘ NHỚ:

- Lỗi tràn số khi tính Bội chung nhỏ nhất (LCM): Công thức $LCM(a, b) = (a * b) / GCD(a, b)$. Nếu thực hiện phép nhân $a * b$ trước, giá trị có thể tràn kiểu `long long` nếu a, b lớn! Cách viết chuẩn: `lcm = (a / gcd(a, b)) * b;`
- Kiểm tra số chính phương: Không dùng phép so sánh số thực `sqrt(n) == (int)sqrt(n)` do sai số dấu phẩy động. Cách an toàn nhất: `long long r = round(sqrt(n)); if (r * r == n) ...`